

CrownCare Analytics Service

File: AnalyticsService.swift

Purpose: Centralized service that emits all analytics events to Firebase according to the schema. Drop-in for your iOS app.

AnalyticsService.swift

```
import Foundation import FirebaseDatabase import FirebaseAuth // MARK: -
Event Models struct AnalyticsEvent { let eventId: String =
UUID().uuidString let eventType: String let userId: String let platform:
String = "ios" let appVersion: String = Bundle.main.appVersion let
sessionId: String let timestamp: TimeInterval =
Date().timeIntervalSince1970 * 1000 let timestampISO: String =
ISO8601DateFormatter().string(from: Date()) let properties: [String: Any]
func toDictionary() -> [String: Any] { return [ "event_id": eventId,
"event_type": eventType, "user_id": userId, "platform": platform,
"app_version": appVersion, "session_id": sessionId, "timestamp":
Int(timestamp), "timestamp_iso": timestampISO, "properties": properties ] }
} // MARK: - Analytics Service class AnalyticsService { static let shared =
AnalyticsService() private let database = Database.database().reference()
private var sessionId: String = UUID().uuidString private var userId:
String? private var eventQueue: [AnalyticsEvent] = [] private var
isProcessingQueue = false private let userDefaultsKey = "crowncare_user_id"
private let sessionStartTime = Date() // MARK: - Initialization init() {
loadOrCreateUserId() setupAppLifecycleListeners() } private func
loadOrCreateUserId() { if let savedUserId =
UserDefaults.standard.string(forKey: userDefaultsKey) { self.userId =
savedUserId } else { let newUserId = UUID().uuidString
UserDefaults.standard.set(newUserId, forKey: userDefaultsKey) self.userId =
newUserId } } private func setupAppLifecycleListeners() {
NotificationCenter.default.addObserver( self, selector:
#selector(appDidBecomeActive), name:
UIApplication.didBecomeActiveNotification, object: nil )
NotificationCenter.default.addObserver( self, selector:
#selector(appDidEnterBackground), name:
UIApplication.didEnterBackgroundNotification, object: nil ) } @objc private
```

```

func appDidBecomeActive() { let isReturningUser = userHasOpenedAppBefore()
let daysSinceLastOpen = calculateDaysSinceLastOpen() trackAppOpen(
isReturningUser: isReturningUser, daysSinceLastOpen: daysSinceLastOpen )
sessionId = UUID().uuidString // New session sessionStartTime = Date() }
@objc private func appDidEnterBackground() { let sessionDurationMs =
Date().timeIntervalSince(sessionStartTime) * 1000
trackAppBackground(sessionDurationMs: Int(sessionDurationMs))
processEventQueue() } // MARK: - Core Tracking Methods func
trackAppOpen(isReturningUser: Bool, daysSinceLastOpen: Int) { let
properties: [String: Any] = [ "session_duration_ms": 0,
"is_returning_user": isReturningUser, "days_since_last_open":
daysSinceLastOpen ] logEvent(type: "app_open", properties: properties)
updateLastOpenDate() } func trackPaywallShown(location: String, tiersShown:
[String]) { let properties: [String: Any] = [ "location": location,
"tiers_shown": tiersShown ] logEvent(type: "paywall_shown", properties:
properties) } func trackSubscriptionInitiated(tier: String, price: Double,
billingPeriod: String) { let properties: [String: Any] = [ "tier": tier,
"tier_price_usd": price, "billing_period": billingPeriod ] logEvent(type:
"subscription_initiated", properties: properties) } func
trackSubscriptionCompleted( tier: String, price: Double, billingPeriod:
String, transactionId: String, receiptData: String, convertedFromTrial:
Bool = false, daysToConversion: Int = 0 ) { let properties: [String: Any] =
[ "tier": tier, "tier_price_usd": price, "billing_period": billingPeriod,
"transaction_id": transactionId, "receipt_data": receiptData,
"converted_from_trial": convertedFromTrial, "days_to_conversion":
daysToConversion ] logEvent(type: "subscription_completed", properties:
properties) linkUserToSubscription(tier: tier) // Convert anonymous to
identified user } func trackFeatureView(featureName: String, category:
String? = nil, timeSpentSeconds: Int = 0) { var properties: [String: Any] =
[ "feature_name": featureName, "time_spent_seconds": timeSpentSeconds ] if
let category = category { properties["category"] = category }
logEvent(type: "feature_view", properties: properties) } func
trackFeatureInteraction( featureName: String, action: String, success: Bool
= true, errorMessage: String? = nil ) { var properties: [String: Any] = [
"feature_name": featureName, "action": action, "success": success ] if let
error = errorMessage { properties["error_message"] = error } logEvent(type:
"feature_interaction", properties: properties) } func
trackSubscriptionCancelled( tier: String, daysSubscribed: Int, reason:
String? = nil, churnCategory: String? = nil ) { var properties: [String:
Any] = [ "tier": tier, "days_subscribed": daysSubscribed ] if let reason =
reason { properties["reason"] = reason } if let churnCategory =
churnCategory { properties["churn_category"] = churnCategory }
logEvent(type: "subscription_cancel", properties: properties) } func
trackAppCrash(error: Error, stackTrace: String? = nil, memoryAvailableMb:
Int = 0) { var properties: [String: Any] = [ "error_message":

```

```

error.localizedDescription, "memory_available_mb": memoryAvailableMb ] if
let stackTrace = stackTrace { properties["stack_trace"] = stackTrace }
logEvent(type: "app_crash", properties: properties) } func
trackAppBackground(sessionDurationMs: Int) { let properties: [String: Any]
= [ "session_duration_ms": sessionDurationMs, "features_accessed_count": 0
// Can be tracked via feature_view events ] logEvent(type:
"app_background", properties: properties) } // MARK: - Private Helper
Methods private func logEvent(type: String, properties: [String: Any]) {
guard let userId = userId else { print("AnalyticsService: No user ID
available") return } let event = AnalyticsEvent( eventType: type, userId:
userId, sessionId: sessionId, properties: properties )
eventQueue.append(event) // Attempt immediate write, queue if offline
writeEvent(event) { [weak self] success in if success {
self?.eventQueue.removeAll { $0.eventId == event.eventId } } } } private
func writeEvent(_ event: AnalyticsEvent, completion: @escaping (Bool) ->
Void) { let dateFormatter = DateFormatter() dateFormatter.dateFormat =
"yyyy-MM-dd" let dateString = dateFormatter.string(from: Date()) let ref =
database .child("analytics") .child("events") .child(dateString)
.child(event.eventId) ref.setValue(event.toDictionary()) { error, _ in if
let error = error { print("AnalyticsService: Failed to write event: \
(error.localizedDescription)") completion(false) } else { completion(true)
} } } private func processEventQueue() { guard !isProcessingQueue else {
return } isProcessingQueue = true let eventsToProcess = eventQueue var
processedCount = 0 for event in eventsToProcess { writeEvent(event) { [weak
self] success in if success { self?.eventQueue.removeAll { $0.eventId ==
event.eventId } } processedCount += 1 if processedCount ==
eventsToProcess.count { self?.isProcessingQueue = false } } } } private
func updateUserProfile( subscriptionStatus: String, tier: String? = nil,
revenue: Double = 0 ) { guard let userId = userId else { return } let
userRef = database .child("analytics") .child("users") .child(userId) var
updates: [String: Any] = [ "subscription_status": subscriptionStatus,
"last_seen": Int(Date().timeIntervalSince1970 * 1000) ] if let tier = tier
{ updates["current_tier"] = tier updates["tier_start_date"] =
Int(Date().timeIntervalSince1970 * 1000) } if revenue > 0 {
updates["lifetime_revenue_usd"] = revenue }
userRef.updateChildValues(updates) } private func
linkUserToSubscription(tier: String) {
updateUserProfile(subscriptionStatus: tier, tier: tier) } private func
userHasOpenedAppBefore() -> Bool { return
UserDefaults.standard.bool(forKey: "has_opened_app_before") } private func
updateLastOpenDate() { UserDefaults.standard.set(true, forKey:
"has_opened_app_before")
UserDefaults.standard.set(Date().timeIntervalSince1970, forKey:
"last_app_open") } private func calculateDaysSinceLastOpen() -> Int { guard
let lastOpen = UserDefaults.standard.value(forKey: "last_app_open") as?

```

```
TimeInterval else { return 0 } let lastOpenDate =
Date(timeIntervalSince1970: lastOpen) return
Calendar.current.dateComponents([.day], from: lastOpenDate, to: Date()).day
?? 0 } } // MARK: - Bundle Extension extension Bundle { var appVersion:
String { return infoDictionary?["CFBundleShortVersionString"] as? String ??
"1.0.0" } }
```

How to Integrate Into Your App

Step 1: Add to SceneDelegate

```
import UIKit class SceneDelegate: UIResponder, UIWindowSceneDelegate { var
window: UIWindow? func scene( _ scene: UIScene, willConnectTo session:
UISceneSession, options connectionOptions: UIScene.ConnectionOptions ) { //
Initialize analytics at app startup _ = AnalyticsService.shared guard let _
= (scene as? UIWindowScene) else { return } } }
```

Step 2: Track Events Throughout Your App

When User Views Paywall

```
// In your PaywallViewController AnalyticsService.shared.trackPaywallShown(
location: "onboarding", tiersShown: ["basic", "premium"] )
```

When User Initiates Subscription

```
// In your SubscriptionManager
AnalyticsService.shared.trackSubscriptionInitiated( tier: "premium", price:
9.99, billingPeriod: "monthly" )
```

When Subscription Completes (StoreKit 2 Receipt)

```
// After successful App Store purchase if let transaction = transaction as?
StoreKit.Transaction { AnalyticsService.shared.trackSubscriptionCompleted(
tier: "premium", price: 9.99, billingPeriod: "monthly", transactionId:
```

```
transaction.id.description, receiptData: receiptData, convertedFromTrial:
false, daysToConversion: daysSinceFirstOpen ) }
```

When User Views a Hairstyle or Feature

```
// In your HairstyleViewController
AnalyticsService.shared.trackFeatureView( featureName: "hairstyle_gallery",
category: "bob_cuts", timeSpentSeconds: 45 )
```

When User Interacts with a Feature

```
// When user saves a hairstyle do { try saveHairstyle(style)
AnalyticsService.shared.trackFeatureInteraction( featureName:
"hairstyle_gallery", action: "save_hairstyle", success: true ) } catch {
AnalyticsService.shared.trackFeatureInteraction( featureName:
"hairstyle_gallery", action: "save_hairstyle", success: false,
errorMessage: error.localizedDescription ) }
```

When User Cancels Subscription

```
// When cancellation is processed
AnalyticsService.shared.trackSubscriptionCancelled( tier: "premium",
daysSubscribed: 45, reason: "Feature gap", churnCategory: "feature_gap" )
```

When App Crashes (In Crash Handler)

```
// In your crash reporting handler AnalyticsService.shared.trackAppCrash(
error: error, stackTrace: crashStackTrace, memoryAvailableMb:
availableMemory )
```

What This Service Does

- **Automatic session management:** Creates a new session ID on each app open, tracks session duration
- **User identification:** Generates and persists a user ID so you can track the same user across sessions

- **Event queuing:** If Firebase write fails, events are queued and retried when the app goes to background
- **App lifecycle integration:** Automatically tracks `app_open` and `app_background` without you doing anything
- **User profile updates:** Automatically updates subscription status and tier in user profiles
- **Timestamp management:** Includes both Unix timestamp (for sorting) and ISO 8601 (for readability)

Important Notes

⚠ **User ID Linking:** After a user subscribes, you should link their anonymous user ID to their account/email. This is done via the `linkUserToSubscription()` method but may need enhancement to store their email in user profiles for your hairstylist dashboard.

✓ **Next Step:** Once you confirm this works, you'll need to create the master analytics dashboard that queries these events. That dashboard will be a separate web app (could be React, Vue, or even a simple HTML page) that reads from Firebase and displays DAU, conversions, revenue, etc.

Bundle Extension: The `Bundle.appVersion` extension automatically reads your app's version from `Info.plist`, so you don't need to manually update it—it's always accurate.

Firebase Rules Update Required

Update your Firebase Realtime Database rules to allow this write pattern:

```
{ "rules": { "analytics": { "events": { ".read": false, ".write": "auth != null" }, "users": { "$user_id": { ".read": "auth != null", ".write": "auth.uid == $user_id || root.child('admins').child(auth.uid).exists()" } } } }
```

File Location

Drop this file in your Xcode project:

```
CrownCare/ ├── Services/ | └─ AnalyticsService.swift ← HERE ├──  
ViewControllers/ ├── Models/ └─ ...
```